Grid Duel
Study Guide

Understand the rules
Trace the code
Prepare for your hearing

A detailed guide in plain language for a second year BSIT student

Based on the current Storm Effects edition

Classic and Vanishing Marks
Game state and turn handling
Minimax and computer difficulty
Lightning magma cracks and electrical surges
Testing practice exercises and hearing questions

Read with the game open. Predict what will happen, try it, and explain the result in your own words.

Reviewed 18 September 2026
Includes a function reference and the complete rules engine

Contents

Click a chapter title in the digital PDF, or use the page numbers in print.

How to use this guide 4

1 What the project actually does 5

2 How the files work together 6

3 JavaScript ideas you need before reading the logic 7

4 How nine squares become an array 9

5 The state is the game memory 10

6 What happens when the page opens 11

7 Trace one move from the click to the next turn 12

8 Legal moves and the order of rule checks 13

9 How the game finds a win or a draw 14

10 Vanishing Marks and the oldest mark queue 15

11 Why repeating positions and long rounds become draws 17

12 Rounds matches scores and settings 18

13 How the computer thinks without machine learning 19

14 Classic AI and full game tree search 20

15 Vanishing AI the horizon and alpha beta pruning 21

16 Difficulty randomness and honest probability claims 23

17 Timers revisions and the event loop 24

18 Rendering the board and making controls accessible 25

19 CSS layout layers and why the last rule matters 26

20 Tension and the three effects settings 28

21 Lightning bolts cracks and board shaking 29

22 How electricity erupts out of the cracks 30

23 Magma smoke embers and procedural textures 32

24 Mark drawing winning lines result stamps and sound 33

25 Testing the game and interpreting the evidence 34

26 Debugging and understanding the current limitations 36

27 Hearing questions with understandable answers 37

28 A practical demonstration for the hearing 40

29 Practice exercises before checking the answers 41

30 Small coding exercises that build real understanding 43

31 A seven session study plan 44

32 Glossary in simple terms 45

33 What to claim and what not to claim 47

Appendix A Function reference and source map 48

Appendix B Read the exact rules engine 52

How to use this guide

You do not need to memorize the entire program. Your first goal is to explain what happens when someone clicks one square. Your next goal is to explain why the move is legal, how a winner is found, and why the computer chooses a particular move. The visual effects become much easier once you understand that foundation.

Keep the game open beside this guide. Read a section, try its example, then explain the result aloud without looking at the page. If you cannot explain a term, check the glossary near the end. Source names in bold point to functions or files you can find with Ctrl+F.

This guide describes the current upgraded copy containing **index.html, style.css, game-core.js, script.js, and verify.cjs**. It does not describe the earlier three-file version as though it were still the current implementation. Small teaching examples are labeled as examples; the full rules engine is reproduced in the final appendix.

A sensible study order: learn Chapters 1 to 11 first, then Chapters 12 to 17, then the effects chapters. Finish with the hearing questions and practice exercises. You can understand the game without first mastering every lighting formula.

1 What the project actually does

Grid Duel is a browser game based on tic-tac-toe. Two sides place X and O on a board with three rows and three columns. A side wins a round by making three marks in one row, column, or diagonal. The first side to win three rounds wins the match.

The project adds a second ruleset called **Vanishing Marks**. Each side can keep only three marks on the board. When that side places a fourth mark, its oldest mark disappears. This means a square that was occupied can become usable again, so the board keeps changing.

Three choices describe different parts of the experience:

Choice	What it changes	Available values
Opponent	Who controls each side	Solo, Two players, AI vs AI
Ruleset	How marks behave	Classic, Vanishing Marks
Difficulty	How the computer chooses moves	Easy, Medium, Hard
Effects	How intense the presentation is	Calm, Vivid, Extreme

Extreme is a visual setting. It does not make the AI smarter, give bonus points, or change which squares are legal. A lightning strike is decorative. It does not destroy a player's mark or affect the winning rules.

In Solo, the human is always X and the AI is O. In Two players, both sides are controlled by people using the same device. This is local multiplayer, not online multiplayer. In AI vs AI, the program controls both sides and pauses after a completed round until someone chooses the next round.

The current project does not include accounts, a database, online matchmaking, cloud storage, or persistent statistics. Reloading the page starts a new session. These are scope boundaries you should explain honestly.

Try it: choose Two players and Classic. Play indices 0, 3, 1, 4, 2 using the board map in Chapter 4. X wins the top row. Notice that the score increases by one, not by three.

Explain it aloud: "Grid Duel keeps the basic three-in-a-row goal, adds a ruleset with disappearing marks, and organizes play into first-to-three matches."

2 How the files work together

Think of the project as a small restaurant. HTML lays out the room and names the stations. CSS controls its appearance. The rules engine decides what is allowed. The controller takes requests, calls the rules engine, and updates what the customer sees. This is an analogy, not an additional feature.

File	Main responsibility	What to study first
index.html	Defines controls, board tiles, labels, and visual containers	Board buttons and script tags
style.css	Defines layout, colors, responsive rules, and animations	.board, .cell, effect classes
game-core.js	Contains rules and computer move selection	create, legal, play, choose
script.js	Connects clicks, game state, score, timers, sound, and effects	makeMove, advanceTurn, restartGame
verify.cjs	Runs automated checks in Node.js	Named test cases
README.md	Explains how to run and verify the game	Play and verification instructions

The browser loads **game-core.js before script.js**. The first file creates an object called **GridRules**. The second file uses GridRules.create(), GridRules.play(), and GridRules.choose(). If you reverse the order, the controller may try to use GridRules before it exists.

The rules engine has no document.querySelector calls and no knowledge of glowing squares. You can run it without a web page. That is why the same file works in Node.js tests.

The last lines of game-core.js choose how to expose the engine. In Node.js, module.exports makes it available to require(). In the browser, root.GridRules places it on the global object. The wrapper function also keeps internal helpers from unnecessarily becoming global names.

This is a useful example of **separation of concerns**: different parts have different jobs. It is reasonable to call the design a separated rules engine and interface controller. Avoid claiming that it is a complete implementation of a formal MVC framework; the project does not use such a framework.

The game runs on the user's device. It does not send each move to an AI service. The computer opponent is a search algorithm written in JavaScript. Claude and Codex helped during development, but neither is contacted to decide moves during gameplay.

Checkpoint: if a win is detected incorrectly, investigate game-core.js. If the board overflows a phone screen, investigate style.css. If a delayed AI move appears after restart, investigate script.js.

3 JavaScript ideas you need before reading the logic

Values and variables

A variable is a name attached to a value. In this game, currentPlayer can contain the text "X" or "O". gameActive contains true or false. moveCount contains a number.

let allows reassignment. **const** prevents reassignment of that variable name. A const array can still have its contents changed; const does not automatically freeze an object.

```
let currentPlayer = "X";
currentPlayer = "O";       // Allowed

const marks = [];
marks.push(4);            // Allowed
// marks = [];            // Not allowed
```

Strings represent text. Arrays represent ordered lists. Objects collect related named properties. Booleans represent yes-or-no conditions. null deliberately means that a result is absent. undefined often means that a lookup or function has not produced a value.

Arrays and objects

An array uses numbered positions starting at zero. board[4] means the fifth item, which represents the center square. An object uses named properties. scores.X means the score associated with X.

```
const board = ["X", "", "", "", "O", "", "", "", ""];
const scores = { X: 1, O: 0 };
```

The empty string "" represents an empty square. It is different from the text "empty". That difference matters because JavaScript treats "" as false-like in conditions.

Functions and return values

A function is a reusable set of instructions. Inputs are parameters; the result can be returned. In other(p), p is the input. The function returns the opposite side.

```
const other = p => p === "X" ? "O" : "X";
```

This arrow function means: if p is exactly "X", return "O"; otherwise return "X". The question mark and colon form a **ternary operator**, a short if/else expression.

return ends the current function. An early return such as "if (!gameActive) return" refuses work immediately. It does not restart the game or exit the entire browser.

Conditions and operators

Syntax	Plain meaning	Example in the game
===	Exactly equal in value and type	player === currentPlayer
!==	Not exactly equal	ticket !== revision

Syntax	Plain meaning	Example in the game
!	Not, or convert a false-like value to true	!gameActive
&&	Both conditions must pass	gameActive && vividEffects
||	At least one condition passes	state.winner or state.draw
>=	Greater than or equal to	moveCount >= 7
++	Add one	revision++
%	Remainder after division	index % 3
...	Spread items or properties into another value	[...s.board]

The actual JavaScript OR operator is two vertical bars. It often returns one of its operands, not necessarily a Boolean. For example, state.winner || state.draw gives the first truthy value. An if statement then interprets that value as true.

Common array operations

push(value) adds an item to the end. **shift()** removes and returns the first item. Together, these implement the mark-age queue. **includes(value)** checks whether a value is present. **every(test)** asks whether all items pass a test. **find(test)** returns the first item that passes, or undefined.

filter(test) keeps matching items. **map(transform)** creates transformed items. **flatMap(transform)** transforms each item and flattens one level; returning [] excludes an item, while returning [i] keeps its index. **forEach(action)** performs an action for each item. **at(-1)** reads the last item.

You do not need to memorize every syntax trick at once. When you see a compact expression, rewrite it as a longer explanation in your own words.

4 How nine squares become an array

The screen shows a two-dimensional grid, but the engine stores a one-dimensional array. A fixed three-by-three board is small enough that one array is simple and practical.

```
Array indices
0 | 1 | 2
--+---+--
3 | 4 | 5
--+---+--
6 | 7 | 8
```

The top-left square is 0, the center is 4, and the bottom-right is 8. A move to index 4 sets board[4] to the current player's symbol.

```
["", "", "", "", "", "", "", "", ""]  // Empty
["", "", "", "", "X", "", "", "", ""] // X at center
```

A row number for internal calculations is Math.floor(index / 3). A column number is index % 3. Both are zero-based. The accessible label adds one because people usually say "row 1" instead of "row 0".

For index 7, Math.floor(7 / 3) is 2 and 7 % 3 is 1. Internally that is row 2, column 1. The user-facing label is row 3, column 2.

In HTML, each cell button has a data-index attribute. JavaScript can read it through cell.dataset.index. HTML attribute values are strings, so renderMark converts the value using Number(...). The click handler also receives a numeric index directly from cells.forEach.

The order of the HTML buttons matters. The current click handler assumes the first selected cell represents index 0, the second represents index 1, and so on. Do not randomly rearrange the HTML buttons without updating the mapping.

Practice: translate this array into a board: ["X", "", "O", "", "X", "", "O", "", ""]. Answer: X is at top-left and center; O is at top-right and bottom-left.

5 The state is the game memory

State means the information needed to describe the game at a particular moment. The screen is a picture of that information. The screen is not the authoritative record of who owns each square.

GridRules.create() builds a new round state:

```
{
  rule: "classic",
  turn: "X",
  board: ["", "", "", "", "", "", "", "", ""],
  queues: { X: [], O: [] },
  moves: 0,
  winner: null,
  draw: null,
  line: null,
  seen: { "X:---------::": 1 }
}
```

Field	Meaning
rule	Either classic or vanish
turn	The side allowed to make the next move
board	The current owner of each of the nine squares
queues	Each side's placed indices, oldest first
moves	Total valid placements in this round
winner	X or O after a win; otherwise null
draw	A reason string after a draw; otherwise null
line	The three winning indices, or null
seen	How often each repetition key has occurred

The queues exist in both rulesets. In Classic they accumulate placement order, but marks are not removed. In Vanishing Marks each queue is limited to three entries.

The controller also keeps match and interface state. **scores** holds round wins. **roundNumber** identifies the round. **roundStarter** says who started it. **roundResults** records completed outcomes. **matchComplete** says that one side has reached three wins.

gameMode, difficulty, ruleset, effectsLevel, and soundEnabled describe settings. **aiTimer, lightningTimer, surgeTimer, and revision** help manage delayed work.

Some values are mirrored: the controller's board, currentPlayer, and moveCount are synchronized from state.board, state.turn, and state.moves after a valid move. They are convenient names for rendering and effects. If you later modify the game, update these consistently; changing only a visible label will not update the rules.

Important distinction: moveCount is a count of placements, not necessarily occupied squares. At move 12 in Vanishing Marks, the board can still contain only six marks.

Explain it aloud: "The state object is the game's memory. A move produces a new state, and the interface redraws the parts that changed."

6 What happens when the page opens

The browser reads the HTML and builds the **DOM**, a tree of elements. The buttons and labels are real browser objects that JavaScript can select.

At the top of script.js, document.querySelectorAll("#board .cell") collects the nine cells. getElementById("status") finds the status paragraph. Similar selections connect the script to scores, settings, and effect layers.

Each cell receives two child spans. The **mark-slot** holds the X or O drawing. The **crack-slot** holds the lightning damage. These are separate so replacing a mark does not erase a crack.

The script creates 16 ember elements and 12 smoke elements. It creates the grain and magma textures once using Canvas. It prepares the result-stamp canvas, installs click and keyboard listeners, and creates the SVG layer used for outward surges.

At the bottom, **syncEffects()** applies the current effects preference and **newMatch()** creates the initial match. The defaults are Solo, Medium difficulty, Classic rules, Vivid effects, sound off, X starting, and scores of zero.

The current version starts play immediately. It still contains some old countdown markup and styling, but there is no active three-second countdown routine in the current round-start flow. Finding an element in HTML does not prove that the feature still runs.

An **event listener** means "when this event happens, run this function." A cell's click listener does not continuously check the mouse. The browser calls it when the player activates that button.

This is **event-driven programming**. Most of the program waits for clicks, key presses, timer callbacks, visibility changes, or media preference changes. It is not one giant loop that constantly recalculates the whole game.

7 Trace one move from the click to the next turn

Use this chapter as your main defense explanation. The most important functions are the cell click handler, **makeMove()**, and **GridRules.play()**.

Step 1 Check who owns the turn

The click listener calls isHumanControlled(currentPlayer). In Two players, this is true for both sides. In Solo, it is true only for X. In AI vs AI, it is false for both.

If a human clicks during the computer's turn, the listener does not call makeMove. This is the first protection against unwanted moves.

Step 2 Check whether the round is active

makeMove(index, player) returns immediately if gameActive is false or player is not currentPlayer. This protects the controller even if a different caller invokes makeMove.

Step 3 Ask the rules engine

```
const next = GridRules.play(state, index);
if (!next) return;
```

GridRules.play checks the square, copies the state, updates the board, and decides whether the result is a win or draw. An invalid move returns null. makeMove leaves the current game untouched in that case.

Step 4 Accept the new state

The controller saves the previous board for comparison, then assigns state = next. It synchronizes board, moveCount, and currentPlayer. The old board remains useful because the engine copied the array instead of editing that old array.

The next state's turn already names the opposite side. The original player parameter still names the side that just moved. This is why the winning score is awarded using player, not currentPlayer.

Step 5 Handle the outcome

If next.winner or next.draw is present, makeMove sets gameActive to false and cancels the AI timer. For a win, it adds one point, checks for three wins, draws the winning line, and changes the message. For a draw, it awards no points.

It adds a round-log entry and changes the main button to Next round or Play again.

If the round continues, it updates the turn message and calls advanceTurn(). If the next side is a human, no AI callback is scheduled. If it is computer-controlled, the controller schedules one.

Step 6 Update what the player sees

renderBoard(previous) changes the marks and labels that need updating. updateTensionEffects() updates the tension-related classes. The move counter shows the new number.

```
Click
  -> human turn check
  -> makeMove guard
  -> GridRules.play
  -> accept new state
  -> win or draw handling OR schedule next turn
  -> redraw board and update effects
```

Plain-language defense answer: "A click does not automatically place a mark. The controller checks turn ownership, the engine checks legality and computes the result, and only then does the interface display the new state."

8 Legal moves and the order of rule checks

GridRules.legal(s) returns the indices of empty squares. If the state already has a winner or draw, it returns an empty list instead.

```
const legal = s =>
  s.winner || s.draw
    ? []
    : s.board.flatMap((v, i) => v ? [] : [i]);
```

For each square, an occupied value returns [], which contributes no index. An empty string returns [i], which contributes that index. The final list might be [1, 3, 5, 7, 8].

GridRules.play(s, i) first checks that i is an integer and that legal(s) contains it. This rejects an occupied square, a finished game, -1, 9, a decimal, null, and even the string "1". The browser normally sends a number, but the engine still checks.

After accepting the index, the engine performs these operations in this order:

1. Copy the old state and its mutable arrays and history object.
2. Remember the moving side as p = s.turn.
3. In Vanishing Marks, remove p's oldest mark if p already has three.
4. Place p at the chosen empty square and append that index to p's queue.
5. Look for a winning line for p.
6. Set turn to the opposite side.
7. Record a win if a line exists.
8. Otherwise, record a draw if the board is full.
9. Otherwise, update repetition history and check repetition and the move limit.
10. Return the new state.

Why copying matters: AI search explores many possible futures. If every hypothetical move modified the real board, the computer would damage the current game while thinking.

The spread operator makes shallow copies. The program explicitly copies the nested board array, both queue arrays, and the seen object because those are the parts it changes. The seen values are numbers, so copying their containing object is sufficient here.

This is not a general-purpose validator for any object someone might invent. The engine expects a sensible state created by its own functions. Its move checks protect the normal game path; they do not validate every possible corrupt state shape.

9 How the game finds a win or a draw

There are exactly eight winning lines on a three-by-three board.

```
[
  [0, 1, 2], [3, 4, 5], [6, 7, 8], // Rows
  [0, 3, 6], [1, 4, 7], [2, 5, 8], // Columns
  [0, 4, 8], [2, 4, 6]             // Diagonals
]
```

The active rules helper is:

```
const win = (b, p) =>
  lines.find(line => line.every(i => b[i] === p));
```

Read it inside out. For one candidate line, every() checks that all three squares contain p. find() returns the first candidate line that passes. If no line passes, it returns undefined. The move function uses "win(n.board, p) || null" to store null instead.

For a top-row X win, the checks are board[0] === "X", board[1] === "X", and board[2] === "X". All three must pass. It is not enough to have any three X marks anywhere.

The returned line is useful twice: it proves the win and tells the interface which cells to highlight. If a move completes two lines, the first matching line in the array is the one stored and drawn. The round still awards one point.

A Classic draw occurs when every square is occupied and no winning line was found. **Winning is checked before a full-board draw**, so a winning ninth move is a win.

Vanishing Marks also uses repetition and a 60-move cap. Its board normally cannot become full because at most three X marks and three O marks remain. The common full-board check is still present in the shared engine.

Example of a Classic draw: indices 0, 1, 2, 4, 3, 5, 7, 6, 8 with X starting produce no three-in-a-row for either side. The score remains unchanged.

The rules depend on array values. Cell colors, lightning damage, animation progress, or the text of the status message do not decide the winner.

10 Vanishing Marks and the oldest mark queue

A **queue** is an ordered list in which the earliest item is handled first. This is called **FIFO**, meaning first in, first out. Imagine a line at a cashier: the person who arrived first is served first.

Each side has its own queue. If X played at 0, then 2, then 4, its queue is [0, 2, 4]. Index 0 is the oldest X mark. The O queue records O's order independently.

The key removal statement is:

```
if (s.rule === 'vanish' && n.queues[p].length === 3)
  n.board[n.queues[p].shift()] = '';
```

Break the compact statement apart:

```
// Equivalent teaching version of the removal step
const oldestIndex = n.queues[p].shift();
n.board[oldestIndex] = '';
```

shift() returns the removed index. That index is then used to clear one board square. Afterward, the engine places the new mark and uses push(i) to put its index at the end of the queue.

A complete seven-move example

Start a new Two players Vanishing round with X starting. Play 0, 1, 2, 3, 4, 5.

Move	Side	Chosen index	X queue afterward	O queue afterward
1	X	0	[0]	[]
2	O	1	[0]	[1]
3	X	2	[0, 2]	[1]
4	O	3	[0, 2]	[1, 3]
5	X	4	[0, 2, 4]	[1, 3]
6	O	5	[0, 2, 4]	[1, 3, 5]

```
After six moves
X | O | X
--+---+--
O | X | O
--+---+--
. | . | .
```

Suppose X chooses index 8. Index 8 is empty, so the move is legal. X already has three marks, so X at 0 disappears. X is placed at 8. The new X queue is [2, 4, 8].

```
After X chooses 8
. | O | X
--+---+--
O | X | O
--+---+--
. | . | X
```

This is **not** an X win. The diagonal 0, 4, 8 is incomplete because 0 has disappeared. Removal happens before the victory check.

Now return to the six-move position and imagine X chooses 6 instead. X at 0 still disappears, but X now occupies 2, 4, 6. That is the opposite diagonal, so X wins.

Can the player choose the oldest occupied square?

No. Legality is checked against the board before removal. In the six-move position, choosing 0 is illegal even though 0 would leave on a valid fourth X placement. The player must first choose an already empty square.

The NEXT OUT label is a warning, not a separate timer. It marks the oldest piece of each side with three marks. O's label does not mean O disappears when X plays. It disappears when O makes O's next valid placement.

Key lesson: the sequence of validation, removal, placement, and win detection is part of the rules. Changing that order creates a different game.

11 Why repeating positions and long rounds become draws

Classic cannot cycle through old boards because marks only accumulate. Vanishing Marks can revisit positions because old marks leave. Without a stopping rule, two players or two AIs could continue the same pattern indefinitely.

The engine creates a text **key** for each position. It includes the side to move, the nine board values, and both queues in order.

```
const key = s =>
  s.turn + ':' +
  s.board.map(v => v || '-').join('') + ':' +
  s.queues.X.join('') + ':' +
  s.queues.O.join('');
```

For the six-move example in Chapter 10, the key is:

```
X:XOXOXO---:024:135
```

"X:" says X moves next. "XOXOXO---" describes the board. "024" is X's queue. "135" is O's queue.

Why include the queues? Suppose the same X marks occupy 0, 2, and 4, but their order is [4, 2, 0]. The picture looks the same, yet a different mark will disappear next. These are different game situations and must have different keys.

The current compact queue encoding is unambiguous because indices are single digits from 0 through 8. If you expand the board to use indices such as 10 or 11, add separators or another structured encoding. Otherwise, different sequences could create the same text.

create() counts the starting position once. After a nonwinning, non-full-board move, play() increments seen[key]. The third occurrence produces the draw reason "Position repeated three times". These occurrences do not need to be consecutive.

The engine also draws at 60 valid placements if there is no win and no earlier draw. That is 60 individual turns, not 60 pairs of turns or 60 seconds.

The order matters: a win on move 60 is still a win. Repetition is checked before the move-cap reason, so if both draw conditions occur together, the displayed reason is repetition.

The AI uses the same play() function for hypothetical Vanishing moves. Its search therefore carries the branch's history and can recognize a draw inside its search horizon.

Practice answer: "We cannot compare only the visible marks for repetition. We also compare whose turn it is and which marks are oldest, because those facts affect the next legal transition."

12 Rounds matches scores and settings

A **round** is one board contest. A **match** is the competition to reach three round wins. The distinction explains why Restart round and New match do different things.

Action	Board	Score	Starter
Restart during a round	Cleared	Preserved	Same as that round
Next round after an outcome	Cleared	Preserved	Alternates
Play again after match completion	Cleared	Reset	X
New match	Cleared	Reset	X
Change opponent, ruleset, or difficulty	Cleared	Reset	X
Change sound or effects	Preserved	Preserved	Unchanged

Clicking an already selected opponent, difficulty, or ruleset returns without restarting. A genuine setting change calls newMatch(). Two players hides the difficulty controls because no AI is choosing moves.

newMatch() sets scores to zero, roundNumber to 1, roundStarter to X, matchComplete to false, and roundResults to an empty list. Then it calls restartGame().

restartGame() increases revision, cancels the AI callback, clears transient effects, creates a fresh engine state using the selected ruleset and roundStarter, resets the interface, and schedules the first AI turn if necessary.

The main button's listener chooses the appropriate behavior. If matchComplete is true, it starts a new match. Otherwise, if the round is finished, it increments roundNumber and flips roundStarter before clearing the board. If the round is still active, it clears the board without changing the starter.

A draw also counts as a completed round for starter alternation. A restart before completion does not.

First to three is more precise than best of five here. At most five decisive rounds are needed, but draws can make the total number of rounds greater than five. For example, X win, draw, O win, X win, draw, X win is a six-round match that ends 3 to 1.

The interface rebuilds the round log from roundResults. A stored winner is displayed as X or O. A null winner represents a draw. The score pips show how many of the three required wins each side has earned.

The engine does not manage the match score. It returns one round's outcome. The controller decides how that outcome affects the match.

13 How the computer thinks without machine learning

The AI does not learn from previous matches. It does not use a neural network, a chatbot, or a training dataset. It explores possible moves and compares their outcomes.

Imagine asking, "If I play here, what is the opponent's best reply? If the opponent replies there, what should I do next?" Repeating this question forms a **game tree**.

A node is a game position. A branch is a possible move. A leaf is either a finished game or the point where the search stops looking ahead.

A **ply** is one player's move. Eight plies could include four moves by each side if the game lasts that long. It does not mean eight full rounds.

The algorithm assumes the opponent will make choices that are bad for the AI. It does not choose a move merely because one possible opponent reply would let the AI win.

A tiny minimax example

Suppose the AI has three candidate moves. After considering the opponent's best defense, their possible results are:

AI choice	Scores available to the opponent	Opponent chooses	Value of the AI choice
A	+10, -10	-10	-10
B	0, 0	0	0
C	+10, 0	0	0

These numbers are teaching values, not the exact constants in the code. The opponent chooses the smallest value because the score is written from the AI's point of view. The AI then chooses the largest of those branch values, which is 0.

This is **minimax**: maximize the AI's outcome while assuming the opponent minimizes it. The algorithm does not average the opponent's replies.

GridRules.choose(state, difficulty) is the entry point. It first finds legal moves. Depending on difficulty, it may return a random legal move. Otherwise, it evaluates candidate moves with the appropriate search.

The same engine handles X or O as the AI. The variable player records which side called choose(). A positive score favors that side, not always X.

Explain it aloud: "The computer simulates both players. It chooses a move with the best result it can secure against a strong reply, rather than choosing a move that only works if the opponent makes a mistake."

14 Classic AI and full game tree search

The nested function **classic(b, turn, left)** searches Classic positions. b is a temporary board, turn identifies the side to move, and left is the remaining search-depth counter used in terminal scoring.

Classic has at most nine placements. The root depth starts at nine, and the function continues until a win or full board. It has no heuristic cutoff. Even from a partly occupied board, the actual board will end before more than nine further placements are needed.

The three stopping conditions

1. If the AI side has won, return 1000 + left.
2. If the opponent has won, return -1000 - left.
3. If the board is full, return 0.

A win is much better than a draw, and a draw is much better than a loss. The left value rewards earlier wins and delays unavoidable losses.

For example, an AI win with left = 6 scores 1006. A later win with left = 2 scores 1002. The AI prefers 1006. A loss with left = 6 scores -1006, worse than a later loss scoring -1002.

Simulate then undo

For every empty square, the function temporarily writes the current side's mark, recursively evaluates the reply, and restores the square to "".

```
b[i] = turn;
const value = classic(b, other(turn), left - 1);
b[i] = '';
```

That final line is essential. Without undoing the move, the next candidate would be tested on a board that still contained the previous candidate's mark.

This mutation is limited to a temporary board created for the search. choose() passes [...n.board] to classic(). It is not freely mutating the live game board.

When turn is the AI's side, best is updated with Math.max. When turn is the opponent, best is updated with Math.min. The initial best value is -Infinity for maximizing and Infinity for minimizing, allowing the first real value to replace it.

Reusing already computed positions

A Map named cache stores previously solved Classic positions. Its key combines the nine board values and the side to move. Before searching a position again, classic() can reuse its stored score.

This is **memoization**. Different move orders can reach the same board, so caching avoids solving the same subproblem repeatedly.

The cache is created inside each choose() call. It is not a permanent database. In a single Classic search, a given occupied board has the same number of placements and therefore the same remaining depth relative to that root. This is why the current cache key does not need to include left. Do not copy this key design blindly to a ruleset with disappearing marks or variable move costs.

Tied moves

Root candidates are checked in this order: center 4, corners 0, 2, 6, 8, then edges 1, 3, 5, 7. The code replaces the chosen move only for a strictly greater score. Equally scored moves keep the earlier candidate.

This ordering explains why Hard often takes the center. It is a tie-breaking preference combined with search, not a rule that the AI must always play center even when another square wins immediately.

Hard Classic is expected to avoid losing from a valid starting game, and the included exhaustive opponent-branch test supports that claim for its selected move policy. It does not mean Hard wins every round. Perfect play from both sides can produce a draw.

15 Vanishing AI the horizon and alpha beta pruning

Vanishing Marks can last longer than nine moves, so fully exploring every continuation would be much more expensive. The current implementation limits its look-ahead.

Hard searches up to **eight plies including its candidate move**. Easy and Medium use up to **four plies on their search branch**. A detected win or draw stops a branch earlier.

After a root candidate is applied, search() receives depth - 1. That is why depth = 8 does not mean eight more moves after the candidate.

Each hypothetical move calls the same **play()** used for real moves. This makes the search respect oldest-mark removal, turn changes, win checks, repetition counts, and the 60-move cap.

Estimating an unfinished position

If the depth limit is reached before the game ends, **evaluate(s, player)** estimates the board. An estimate is called a **heuristic**.

For each winning line, the program counts the AI's marks and the opponent's marks. A line containing both sides cannot become a three-in-a-row without future removals, so it contributes zero under this simple current-board estimate.

For a line with no opponent marks, the code adds a value based on the AI mark count. For a line with no AI marks, it subtracts a value based on the opponent count.

Marks belonging to one side in an open line	Magnitude
0	0
1	1
2	7
3	100

An open line with two AI marks contributes +7. An open line with two opponent marks contributes -7. A blocked line contributes zero. Values across all eight lines are added.

For a concrete unfinished Classic-shaped board where X occupies 0 and 1 and O occupies 4, evaluate(s, "X") returns 6: top row +7, middle row -1, left column +1, middle column blocked, one diagonal blocked, other diagonal -1, remaining lines zero. This example demonstrates the arithmetic; the same evaluator is used at Vanishing search leaves.

A real win returns a terminal score around +1000 before the heuristic is reached. The heuristic's +100 table entry should not be described as the actual reward for winning the round.

What the estimate misses

The heuristic looks at the current board; it does not directly assign extra penalties based on which mark is about to expire. Queue order still affects the simulated moves before the cutoff. At the final cutoff, however, the estimate is limited.

A threat one move beyond the search horizon can be missed. This is a reason Hard Vanishing is strong but not guaranteed unbeatable.

Alpha beta pruning in simple terms

Alpha-beta pruning skips search work that cannot change a decision already supported by another option.

Suppose the AI has already found a candidate that guarantees at least 4. While considering another candidate, it discovers that the opponent can force its score down to 1. There is no need to examine every other reply under that candidate merely to prove it is not better than 4.

alpha tracks the best lower bound available to the maximizing side along the current search. **beta** tracks the best upper bound available from minimizing choices. When beta <= alpha, the remaining siblings at that point can be skipped.

This does not mean the algorithm randomly ignores dangerous moves. Under the same depth and evaluation rules, correctly applied pruning preserves the minimax result while reducing the amount of work.

The Vanishing search uses alpha-beta pruning but does not use the Classic cache. A safe Vanishing cache would need to account for queue order and draw-relevant history or otherwise establish which information can be safely ignored.

16 Difficulty randomness and honest probability claims

Difficulty changes how often the AI selects a random legal move and how far its Vanishing search looks ahead.

Difficulty	Random branch	Search branch	Vanishing search depth
Easy	70 percent	30 percent	4 plies
Medium	20 percent	80 percent	4 plies
Hard	None	Always	8 plies

In Classic, every difficulty's search branch uses the complete Classic solver. Easy and Medium do not use a deliberately shallower Classic solver. Their weaker play comes from choosing random moves more often.

A random move is still legal. The program selects an index from the legal-move list. It does not choose an arbitrary number and hope the square is empty.

These percentages are branch probabilities for each decision, not win rates. A random move may happen to be the best move. Easy therefore does not make a mistake on exactly 70 percent of turns, and Medium does not have an 80 percent chance of winning.

The choose() function accepts an optional random function, defaulting to Math.random. Passing a controlled random source can make a test reproducible. The simulation test also uses a seeded random generator so its 1,000 rounds can be repeated.

Randomness in a lightning pattern is separate from the meaning of game rules. Decorative randomness affects appearance. AI randomness affects which legal move is selected.

Complexity in simple terms: if each position has b choices and you look d moves ahead, naive search can grow roughly like b to the power d. Alpha-beta and caching reduce work in suitable cases, but they do not make all game searches instantly cheap. The fixed nine-square board makes this project manageable.

Do not use "O(1)" for the whole AI merely because the visible board size is fixed. For explaining the algorithm, discuss the branching factor, depth, terminal conditions, and search optimizations. For the win check itself, checking eight fixed triples is a small constant amount of work in this specific game.

17 Timers revisions and the event loop

The game deliberately waits before an AI move so the player can see the turn change. **advanceTurn()** schedules the AI after 380 milliseconds in Solo or 650 milliseconds in AI vs AI.

A millisecond is one-thousandth of a second. These delays are about 0.38 and 0.65 seconds. setTimeout does not guarantee an exact execution time; it makes a callback eligible after at least the requested delay, subject to browser scheduling.

A **callback** is a function passed to another function to run later. The callback remembers the variables from where it was created. This remembered surrounding context is a **closure**.

The restart problem

Imagine X moves and an AI callback is scheduled. Before the delay expires, the player restarts. If the old callback is allowed to run, it might place an O on the new board.

The current controller uses several protections:

1. clearTimeout(aiTimer) cancels the stored pending callback.
2. restartGame() increments revision.
3. advanceTurn() stores the current revision in a local ticket.
4. The callback checks ticket !== revision before doing anything.
5. It also checks gameActive, currentPlayer, human ownership, and document.hidden.

```
const ticket = revision;
const player = currentPlayer;

aiTimer = setTimeout(() => {
  aiTimer = null;
  if (ticket !== revision ||
      !gameActive ||
      currentPlayer !== player ||
      isHumanControlled(player) ||
      document.hidden) return;

  const move = GridRules.choose(state, difficulty);
  if (move !== null) makeMove(move, player);
}, gameMode === 'cvc' ? 650 : 380);
```

Think of revision as the round's version stamp. A ticket from version 4 is rejected after restart changes the current version to 5. Checking only "is it O's turn?" is weaker because O might also happen to be the next player in a different round.

JavaScript usually runs these page callbacks on one main thread. The problem is stale scheduled work, not two pieces of JavaScript literally editing the board at the same instant.

Hiding the page

On visibilitychange, the controller clears the AI timer and stops lightning and surges. It also adds effects-paused while hidden. On return, if the round is active, it refreshes status and effects and schedules the current AI side again.

The AI calculation itself is synchronous once started. A timeout postpones its start; it does not move the search into a background worker. A Web Worker would be a possible improvement if a larger game made the search slow.

Explain it aloud: "We cancel pending work and also verify a revision ticket. This prevents a move scheduled for an old round from being accepted in a new one."

18 Rendering the board and making controls accessible

Rendering means turning data into something visible. renderBoard(previous) reads the current board and updates the cell elements.

It compares previous[i] with board[i]. If the value changed, it clears that cell's mark-slot and inserts the appropriate X or O SVG. It does not need to replace all nine marks after every move. Recreating unchanged marks would unnecessarily restart their drawing animations.

It toggles classes such as mark-x, mark-o, vanished, expiring, and last-move. A class is a named label CSS can use to style an element.

For Vanishing Marks, a cell is expiring when the round is active, the cell contains a mark, that side has three marks, and the cell is first in that side's queue. Both players' oldest marks can be labeled at the same time.

The last-move indicator reads the last index in the previous mover's queue. After play(), currentPlayer is already the next side, so the previous mover is the opposite side.

Why aria-disabled is used

The code keeps cell.disabled = false and sets aria-disabled to describe cells that cannot currently be played. That keeps the tiles reachable by keyboard so a user can inspect their labels.

aria-disabled does not block JavaScript clicks by itself. The controller and engine must still reject illegal moves. Accessibility attributes describe behavior; they are not a substitute for validation.

Each cell receives a label such as "Row 2, column 3, O" or "Row 1, column 1, X, next to vanish for X". The status paragraph has role="status" and aria-live="polite", allowing assistive technology to announce changes without an urgent interruption. aria-atomic="true" requests the entire status message as one update.

The board's aria-busy indicates that an active computer turn is in progress. Decorative overlays use aria-hidden so they do not add meaningless narration.

Keyboard navigation

Tab moves through normal controls. Enter or Space activates a focused button. The board additionally supports arrow keys.

Left and Right wrap within the current row. From index 0, Left moves focus to 2. Up and Down move by three indices with wraparound; from index 1, Up moves to 7.

This behavior uses remainders. For example, (index + shift + 9) % 9 keeps a vertical destination within 0 through 8. The added 9 avoids a negative result for the small negative shift used here.

The key handler calls event.preventDefault() so an arrow key used for board navigation does not also scroll the page.

These features improve keyboard and screen-reader usability, but they do not establish that a complete accessibility audit has been performed.

19 CSS layout layers and why the last rule matters

CSS controls appearance through selectors and properties. A selector such as .cell targets elements with that class. A selector such as body.extreme-effects .surge-core targets surge-core elements when the body has the extreme-effects class.

The browser's **cascade** determines which competing declaration wins. Specificity, importance, and source order all matter. When comparable rules conflict, the later applicable rule usually takes precedence. A declaration with !important changes the usual priority.

The current style.css contains the original style foundation followed by the Duel Edition refinements and the Storm Effects additions. Therefore, reading only an early rule can give you the wrong idea about the final behavior.

For example, older styles shake #board using tension classes. The updated design neutralizes that older movement and applies the active tremor to **.board-wrap**. That wrapper contains both the tiles and the surge overlay, allowing them to move together.

The board layout

The board uses CSS Grid with three equal columns and three equal rows. aspect-ratio keeps it square. Gaps between cells reveal the magma layer underneath.

The desktop arena has settings and the play area beside each other. At 760 pixels or narrower, the arena becomes a vertical layout. The game board is visually placed before the settings so phone users can start playing without scrolling past the configuration.

The viewport meta tag helps the browser use the device width rather than a larger default layout viewport. Breakpoints change presentation, not rules.

Important visual layers

Layer	Purpose
Page background and texture	Dark arena atmosphere
Lava layer under the board	Magma visible in the tile gaps
Cell surface	The physical-looking tile
crack-slot inside each cell	Persistent fracture art

Layer	Purpose
mark-slot inside each cell	X or O
Winning line SVG	Shows the winning triple
Surge SVG over the board	Arcs rising beyond tile edges
Fixed fx overlay	Screen-space bolt, result stamp, and celebration particles

z-index helps order layers, but stacking contexts also matter. Clipping, transforms, positioned elements, and isolation can influence how layers are grouped. Do not explain z-index as one universal number that always beats every element on the page.

The clipped cell shape is why the new outward surges live outside individual cells. An arc drawn entirely inside a clipped cell could be cut off at the cell boundary.

pointer-events: none on decorative overlays lets clicks pass through to the board. A bright effect should not steal the user's input.

20 Tension and the three effects settings

The tension level is determined by total valid placements in the current round.

Moves	Internal tier	Meaning
0 to 2	calm	Opening
3 to 4	mid	Early buildup
5 to 6	high	Lightning and surge systems can run
7 and above	critical	Strongest tension tier

tensionTier() checks the largest threshold first. If it checked "at least 3" before "at least 7", a seven-move position could be returned as mid too early.

The tension meter has nine segments. Segment i lights when i < moveCount. At 12 Vanishing moves, all nine segments are lit, while the numeric move label still shows 12. The meter saturates; it is not a percentage of how many squares are occupied.

Tension is not an AI evaluation. It does not know whether a player is about to win. It is a presentation measure based on move count.

Setting	Board motion	Lightning and surges	Magma
Calm	Steady	Disabled and cracks cleared on selection	Subtle presentation
Vivid	High and critical tremors	Active from high tension	Animated, brightening seams
Extreme	Tremor begins at mid	More branches, arcs, and frequent strikes	Faster, stronger activity

Vivid is the default in the current edition. Sound is a separate control and starts off.

The motion preference override

effectsLevel stores the user's selected option. vividEffects is a Boolean meaning animated storm effects are currently allowed. It is true only when the chosen level is not Calm and the system does not request reduced motion.

A user can leave the selector on Extreme while the system preference disables its motion. The explanatory reduced-motion note makes this visible.

syncEffects() stops existing lightning and surges, changes body classes, updates the control, clears cracks if animated storm effects are disabled, and reapplies appropriate tension effects for an active round.

At round end, the controller uses a calm effective tier and stops the active storm timers. Existing crack artwork can remain visible on the finished board until the next reset or Calm selection.

21 Lightning bolts cracks and board shaking

A lightning strike has a target

startLightning() prevents duplicate scheduling by checking lightningTimer. When the storm first starts, it strikes immediately, schedules the next strike, and starts recurring surges.

pickStrikeTarget() first collects cells that are not already cracked. It chooses randomly among them. Once every cell is cracked, it can choose any of the nine cells.

strikeBoltOnBoard() reads the chosen cell's rectangle with getBoundingClientRect(). The target is the center: left plus half the width, top plus half the height. The screen-space bolt SVG uses the current viewport dimensions.

Drawing a jagged path

buildBoltPath() starts above the screen at y = -30. It chooses a starting x near the target, then creates a sequence of points leading down to the cell.

The fraction t = i / segments expresses how far the path has progressed. At the beginning t is near zero; at the last point it is one.

Random horizontal jitter makes the bolt jagged. The jitter is multiplied by (1 - t) raised to 0.7. Near the target this becomes smaller, and at t = 1 it is zero. The path therefore converges to the intended impact point.

Vivid uses 14 path segments. Extreme uses 19 and a wider possible jitter range. buildBranches() creates two shorter forks from intermediate points. A bright central path and a wider blurred path produce the layered glow.

This is procedural drawing, not a physical simulation of electricity.

Cracks stay on their tiles

impactCell() generates a crack SVG, stores its vent endpoints, and replaces the chosen cell's crack-slot content. Replacing rather than appending keeps repeated strikes from piling up unlimited fracture elements.

The crack starts at coordinates 50, 50 in a 100-by-100 local viewBox. Each arm takes three jagged steps. Coordinates are clamped between 7 and 93 so the arm stays inside the tile's drawing area.

Vivid generates three arms; Extreme generates five. The returned endpoint of each arm becomes a possible source for an outward surge.

Math.cos(angle) and Math.sin(angle) convert an angle into horizontal and vertical directions. You can explain this simply as choosing which way each crack branch travels.

Shaking intensity

Setting and tier	Nominal movement variable	Cycle duration
Vivid high	1 pixel	0.30 seconds

Setting and tier	Nominal movement variable	Cycle duration
Vivid critical	1.5 pixels	0.19 seconds
Extreme mid	0.6 pixels	0.40 seconds
Extreme high	2 pixels	0.18 seconds
Extreme critical	3 pixels	0.13 seconds

The keyframes multiply this variable by different factors, so it is not a claim that every point moves exactly that many pixels. Shorter cycles feel faster. The board wrapper moves as one unit. Separate brief arena jolts accompany strikes.

The motion never changes board indices or legal moves. It changes the rendering transform.

22 How electricity erupts out of the cracks

The important visual change is a separate SVG called **surgeLayer**, placed over the board wrapper. It allows arcs to extend past tile edges instead of staying trapped in the crack-slot.

crackVents is a Map from cell index to an array of fracture endpoints. A Map is useful because the program can look up the stored endpoints for a particular cell.

The sequence is:

1. A lightning impact calls buildCrackSvg().
2. That function returns the fracture artwork and endpoint coordinates.
3. impactCell() stores the endpoints in crackVents.
4. emitSurge(index) chooses endpoints and creates paths above the tile.
5. CSS animates the paths, particles, and small vent rings.
6. A cleanup timer removes the temporary SVG group.

Converting coordinates

A vent endpoint is expressed as a percentage-like coordinate in the cell's 100-by-100 drawing space. The overlay needs a location relative to the board wrapper.

The x calculation combines the cell's left offset from the wrapper with tile.width multiplied by vent.x / 100. The y calculation does the same for vertical position. A scale factor adjusts for the measured wrapper width.

Subtracting the wrapper's position removes the whole-page offset. Moving the wrapper and overlay together keeps the arcs attached during a tremor.

Building an upward arc

Each arc starts at a vent. Its y coordinate decreases as the path grows, so it travels upward. A random sideways spread and intermediate jitter keep it from looking like a straight pipe.

Each main eruption has seven path steps and a small branch. One path is a blurred blue halo; another is the narrower bright core. Their shared geometry makes the glow appear attached to the electrical stroke.

A small ellipse expands and fades at the source. Tiny circles receive random --dx and --dy values, which CSS uses to move particles outward and upward.

Parameter	Vivid	Extreme
Crack arms per new impact	3	5
Arcs in each surge group	2	4
Particles per arc	3	5
Recurring surge delay	650 to under 1100 ms	280 to under 540 ms
High-tier strike delay	2300 to under 3400 ms	1380 to under 2040 ms
Critical-tier strike delay	1500 to under 2400 ms	900 to under 1440 ms

The upper limits are normally exclusive because Math.random() is less than one. Actual callback timing can be later because of browser scheduling.

Preventing buildup

surgeCleanups maps each active group to its removal timer. emitSurge() refuses to add another group when six are already tracked. Each group is removed after 850 ms. Its main visual animation finishes sooner, leaving a small cleanup margin.

stopSurges() clears the recurring timer, clears all group-removal timers, empties the Map, and removes the group's DOM elements.

This cleanup runs through stopLightning(), including when the round ends, the page is hidden, a restart occurs, or the effects preference changes. Switching to Calm also clears the stored cracks.

These limits control object buildup. They do not prove a fixed frame rate on every device.

23 Magma smoke embers and procedural textures

Procedural means generated using code instead of loaded from a pre-made picture. The game draws a small magma texture using Canvas, then uses that texture in CSS.

generateLavaTexture() creates a 180-by-180 canvas. It paints a dark rock base, adds 30 warm glowing spots, covers parts with 16 irregular dark plates, adds nine jagged hot veins, and applies fine pixel variation.

A radial gradient starts bright near a center and fades outward. An ellipse stretches a circle. Irregular polygons make the rock plates less uniform. Random variation prevents every generated texture from looking exactly the same.

After drawing, canvas.toDataURL() turns the image into a data URL. JavaScript places it into the --lava-texture CSS variable. CSS can then use it as a background image without requesting an external image file.

The canvas texture is generated once during page setup. The game is not repainting that whole texture with JavaScript on every animation frame.

Where the movement comes from

The lava layer combines the generated texture with an animated warm gradient. CSS moves the gradient background to suggest flowing heat.

The Storm Effects rules add two pseudo-elements. One contains glowing radial plumes that move, rotate, and scale. The other contains repeating bright points that move upward. The board tiles hide most of this layer, so the motion is seen mainly through the seams.

The result is a visual illusion of magma circulation. The code does not calculate fluid pressure, temperature, or real rock fracture physics.

Embers and smoke

initEmbers() creates 16 small elements with randomized horizontal positions, durations, and delays. initSmoke() creates 12 puffs with randomized drifting behavior.

CSS classes determine which of those pre-created elements animate at a given tension tier. Higher tiers increase activity. Calm and reduced-motion rules limit or hide these moving decorations.

The page also generates a small noise texture using a 96-by-96 canvas. It changes pixel brightness and transparency slightly, then applies the texture at low opacity to the background.

Using a limited number of reusable elements is more controlled than endlessly creating new particles. The separate surge system is temporary, but it has a strict active-group cap and cleanup.

24 Mark drawing winning lines result stamps and sound

Drawing X and O

The marks are SVG shapes. X contains two lines, and O contains a circle. CSS uses stroke-dasharray and stroke-dashoffset to make the stroke appear to draw itself. Reducing the dash offset reveals more of the stroke.

These are shapes, not a special font character. SVG scales cleanly as the board size changes.

Drawing the winning line

The engine returns three winning indices. drawWinLine() finds the center of the first and last cells using cellCenterPercent(). It maps those positions into the winning-line SVG's 0-to-100 coordinate system.

Using measured cell positions accounts for actual padding and gaps. A resize listener recalculates the line if the state contains a winner.

The winner color comes from TEAM_COLOR: blue for X and orange for O. Winning tiles are marked with a CSS class, and the line is animated into place.

Result stamp and celebration

playStamp() draws text such as YOU WIN, X WINS, or DRAW on a canvas. The canvas uses devicePixelRatio when sizing its internal pixel buffer, helping it look sharper on high-density screens.

After 950 ms, dissolveStamp() clears small blocks in a shuffled order. requestAnimationFrame schedules steps in coordination with the browser's rendering cycle. The effect targets about 34 steps, not a guaranteed fixed duration on every screen.

clearStamp() cancels the pending dissolve timer and animation-frame callback and clears the canvas. That prevents an old result effect from carrying into a new round.

A win also creates 22 short-lived celebration sparks. They remove themselves after their CSS animation ends and have a one-second fallback removal timer. Reduced motion skips their creation.

Calm means a steady, less intense presentation; it does not mean every result animation in the entire game is turned off. System reduced motion is the broader motion override.

Synthesized audio

The game does not load recorded sound files. It uses the Web Audio API to make sine-wave tones.

An oscillator generates the tone. Its frequency controls pitch. A GainNode controls volume. The code connects oscillator to gain and gain to the audio destination.

The volume rises briefly, then fades to a very small value. Each oscillator stops after about 0.18 seconds, and the connected nodes are disconnected when it ends.

X moves use 520 Hz and O moves use 390 Hz. A win uses a short sequence of 440, 554, and 659 Hz. A draw uses 260 and 220 Hz. The notes in a sequence start 0.09 seconds apart.

The AudioContext is created only when sound is enabled and requested. Browsers may require a user gesture before audio starts. If the context needs resuming, the current implementation requests resume and returns; the first attempted tone may therefore be skipped.

Explain it aloud: "The marks and bolts are SVG, textures and result text use Canvas, and the sound is generated with oscillators. The browser runs all of these locally."

25 Testing the game and interpreting the evidence

A test compares actual behavior with an expected result. If the values do not match, an assertion fails. A good test asks a meaningful question, such as whether an expired mark is removed before a winning line is evaluated.

The current **verify.cjs** file uses Node's built-in assert module. It imports the exact game-core.js used by the browser.

Run it from the game folder:

```
node verify.cjs
```

Node.js is needed to run this test command. It is not required for someone who only opens index.html to play.

The eight automated suites

Suite	What it checks
Classic results	A known win, known draw, and refusal of moves after a result
State safety	Illegal inputs return null and the original state is unchanged
Vanishing order	The oldest mark leaves before win detection; occupied oldest squares remain illegal
Repetition	The third recorded position draws and queue order changes the key
Move cap	Move 60 can draw, while a win on that move has priority
Hard Classic	Every legal opponent branch under the AI's chosen strategy avoids an AI loss
Vanishing AI	The AI finds a known immediate win without changing its input
Randomized rounds	1,000 reproducible Vanishing games maintain invariants and terminate

An **invariant** is a fact that should remain true after every valid move. In the randomized tests, each queue stays at most three entries long, each queue index belongs to that side on the board, and board counts agree with queue lengths.

The random simulation continues until a winner or draw. It checks that the move count stays within 60. Random testing explores a variety of games, but it does not exhaust every possible Vanishing position or prove that its AI is optimal.

The Hard Classic test tries both starting players and both AI symbols. On the AI's turn it follows the AI's chosen move. On the opponent's turn it explores every legal response. A Set avoids repeatedly exploring the same board and turn.

The recorded run reports **422 terminal outcomes** for that search policy. This is not the total number of tic-tac-toe games and not 422 human playtest participants.

Some tests deliberately prepare internal conditions. For example, the repetition test sets the count for the next position to two before making the third occurrence. This isolates the rule, but it is not the same as manually playing a complete looping sequence in the browser.

Browser checks

Browser testing covered full matches, rematches, draws, disappearing marks, keyboard arrows, settings changes while an AI move was pending, and AI vs AI scheduling.

The effects update was checked for Vivid and Extreme activation, round-end cleanup, restart cleanup, changing to Calm, live reduced-motion changes, and mobile layouts. No JavaScript page errors were observed in those checks.

The shaking board is intentionally moving. Browser automation tools may wait for a target to be stable before clicking it, so the effects tests used forced pointer clicks while the board was moving. This is a testing-tool issue, not evidence that the moving cells became unclickable.

These checks support confidence in the tested behavior. They do not establish zero bugs, perfect accessibility, measured educational benefit, or a fixed frame rate on every device.

26 Debugging and understanding the current limitations

Use a repeatable method instead of randomly editing code.

1. Describe exactly what went wrong.
2. Write the shortest sequence that reproduces it.
3. Identify whether the problem belongs to rules, controller, rendering, or effects.
4. Inspect the relevant values before and after the failing action.
5. Change one cause at a time.
6. Repeat the original sequence and related regression tests.

Safe inspection examples

These examples are for the browser's developer console while the game is open. Opening developer tools and reading values does not modify the game.

```
state.board
state.turn
state.queues
state.moves
state.winner
state.draw
scores
revision
```

To investigate timers and effects, inspect aiTimer, lightningTimer, surgeTimer, and surgeCleanups.size. A null timer variable means the controller is not currently tracking a pending timer of that kind.

Avoid pasting commands that deliberately overwrite state unless you are working on a disposable test copy. The normal interface maintains several related values together.

Useful breakpoints

Place a breakpoint at the start of makeMove() to inspect the chosen index and moving player. Step into GridRules.play() to see the copy, queue removal, placement, and result checks. Continue to renderBoard() to see how data becomes visible.

A breakpoint pauses JavaScript execution. Real time may pass while paused, so animation and timer behavior during debugging can differ from normal use.

Boundaries worth knowing

The game has no persistent save system. There is no server that verifies moves and no online multiplayer protocol. Both local players can inspect the same device, so this is not a secure competitive online platform.

The AI runs on the main thread. A larger board or much deeper Vanishing search could cause visible pauses. Extreme effects may cost more to render on slower hardware.

The heuristic does not fully model mark age at its cutoff. The tension meter measures move count, not danger or player emotion. The game's accessibility features are useful, but no full formal audit is claimed.

The source retains some older code and styles. Examples include countdown markup, a countdown-number reference, the unused controller winningPatterns array, and earlier CSS rules that are overridden later. The active winning patterns are the lines in game-core.js.

These leftovers are maintenance opportunities. Do not describe the unused countdown as a working feature. Before deleting anything in future work, search for its references and rerun the relevant checks.

27 Hearing questions with understandable answers

Use these as models for understanding, not as a script for pretending to know something. Rewrite the answers in your natural speaking style.

What is your project

"Grid Duel is a browser-based tic-tac-toe game with Classic and Vanishing Marks rules, local player and AI modes, and first-to-three matches. It also uses visual effects to represent increasing tension."

What did you add beyond ordinary tic-tac-toe

"The main gameplay addition is Vanishing Marks. Each player keeps only three marks, so their oldest mark disappears when they place another. I also have match scoring, alternate starters, AI choices, and adjustable visual effects."

Why did you choose this project

Give your real reason. A possible truthful explanation, if it matches your experience, is: "The rules are easy to demonstrate, but the implementation let me study state management, AI search, event handling, and interactive graphics." Do not invent surveys or user interviews.

What technologies does it use

"The page structure is HTML, the appearance and animations are CSS, and the behavior is JavaScript. The browser provides SVG, Canvas, and Web Audio. Node.js is used to run the rules tests."

Where is the board stored

"In a nine-item array. Each item is X, O, or an empty string. Array indices 0 through 8 map to the visible squares."

How do you detect a winner

"I store the eight possible winning triples. For each triple, I check whether all three indices belong to the moving player. The first matching triple is returned and used for the visual winning line."

Why does a player not win before an old mark disappears

"In Vanishing Marks, removal is part of the move. The engine removes the oldest mark, places the new one, and then checks the resulting board. Checking earlier would count a mark that no longer exists."

What data structure controls disappearing marks

"Each player has a FIFO queue implemented using an array. shift removes the oldest index, and push adds the newly placed index."

Why must a move target an already empty square

"That is the rule implemented by the legality check. The chosen square is validated before removal. A player cannot select their occupied oldest square as a special replacement move."

How do you stop endless Vanishing rounds

"The engine draws on the third occurrence of an equivalent position or after 60 valid moves. The position key includes the board, whose turn it is, and both players' mark order."

Is your AI machine learning

"No. The running game uses search-based AI. It simulates possible moves and evaluates outcomes. It does not train a model or ask a chatbot for moves."

What is minimax

"It chooses the best outcome for the computer while assuming the opponent will choose the reply that is worst for the computer."

What is alpha beta pruning

"It skips branches that cannot improve a decision already supported by another option. It reduces work without changing the minimax result at the same search depth and evaluation."

Why is Hard Classic different from Hard Vanishing

"Classic has a small game tree that ends when the board fills or someone wins, so the solver searches to the end. Vanishing can continue much longer, so Hard looks eight individual moves ahead and estimates unfinished positions."

Does Hard always win

"No. Hard Classic can draw against strong play. Hard Vanishing can lose because its look-ahead is limited. Unbeatable does not mean guaranteed to win."

How do Easy and Medium work

"Easy chooses a random legal move 70 percent of the time; Medium does so 20 percent of the time. Otherwise they search. Those are decision probabilities, not measured win rates."

What happens when someone restarts during an AI turn

"The pending timeout is cancelled and the round revision changes. The callback also checks its saved revision, turn ownership, and active-round status before accepting a move."

Why split the rules from the interface

"So I can test the game rules without needing a browser. The controller can change the presentation without changing how a legal move is calculated."

Does lightning damage the board logically

"No. Cracks and electricity are visual. They do not remove marks, block squares, change turns, or award points."

How do the surges come out of the cracks

"The crack generator returns endpoint coordinates. The controller converts them into board-relative positions and draws separate SVG paths above the tiles. That overlay is outside the clipped cells, so the arcs can extend into the gaps and air."

How do you limit the effect workload

"Surge groups are capped at six and removed after 850 milliseconds. Recurring timers and existing groups are cleared on reset and other stopping conditions. Crack artwork is replaced rather than repeatedly appended."

Why provide Calm and reduced motion

"Players have different motion preferences and devices. Calm gives a steady board. A system reduced-motion preference overrides animated storm effects even if Extreme remains selected."

How did you test it

"The rules have eight automated suites, including invalid moves, removal ordering, repetition, and 1,000 simulated Vanishing rounds. The Classic AI was checked against all legal opponent responses under its chosen strategy. Browser checks cover the interface and effects separately."

What are the main limitations

"There is no online multiplayer or persistent storage. The Vanishing heuristic is limited, effects may cost more on slow devices, and the testing does not cover every browser or prove that the game improves learning."

Did you use AI tools to develop it

"Yes. Claude and Codex assisted with development. I am studying and verifying the code so I can explain how it works." Add only contributions you actually performed and understood. Follow your institution's disclosure requirements.

What would you improve next

Choose a focused extension you can explain: save preferences locally, add stronger Vanishing evaluation, move expensive search into a worker, or improve source organization. Explain the work it would require rather than claiming it is already implemented.

28 A practical demonstration for the hearing

Practice this sequence before presenting. Use Two players so you control both sides and do not depend on random AI choices.

Demonstration 1 Classic rules and scoring

Start a new Classic match. Point to the index map if the panel needs it. Play 0, 3, 1, 4, 2. Explain that X completes [0, 1, 2], the engine returns that triple, and the controller awards one point.

Try an empty cell after the win. Explain that a finished state has no legal moves. Choose Next round and show that O starts. Restart during that new round and show that O remains the starter.

Demonstration 2 A surprising Vanishing move

Switch to Vanishing Marks, which starts a new match. Play 0, 1, 2, 3, 4, 5. Point out the NEXT OUT labels. Play X at 8 and explain why the apparent 0, 4, 8 diagonal does not win: 0 has already left.

Start a new match and repeat the first six moves. This time play X at 6. Explain that the surviving X marks are 2, 4, 6, a valid diagonal.

Demonstration 3 Visual and logical separation

Use a fresh round and build to high tension without an early win. Change Vivid to Extreme and explain that it changes branching, timing, and motion. Change to Calm and show that the board marks and score remain the same.

Keep this section brief. The graphics demonstrate interface work; the rule trace demonstrates the programming foundation.

Demonstration 4 AI and test evidence

Choose Solo and Hard Classic. Explain minimax before playing a move. Do not promise a particular human-versus-AI game will be a win. Show that the AI replies legally.

If Node.js is available, run node verify.cjs and explain one test in detail. Explaining one assertion accurately is more convincing than only pointing to a page full of PASS messages.

If something unexpected happens

State what you observed, reproduce it if possible, and explain which part of the program you would inspect. Do not call every unexpected result a bug: the first Vanishing example is intentionally not a win.

Keep a known working copy for presentation. Avoid changing code live unless the panel specifically asks and you understand the change.

29 Practice exercises before checking the answers

1. On the index map, identify the center, bottom-left, and middle-right squares.
2. After X plays 0, O plays 4, and X plays 1 in Classic, write the board array and identify the next side.
3. Why does a full board not automatically mean a draw?
4. In the six-move Vanishing position from Chapter 10, what happens if X tries to choose 0?
5. In that same position, X legally chooses 8. Write the new X queue and identify the emptied square.
6. Explain why two identical-looking boards can have different repetition keys.
7. A round has score X = 2 and O = 1. O wins the round. What is the new score, and is the match finished?
8. In Solo, O is thinking and the player restarts. Name two mechanisms that prevent the old move from leaking into the new round.
9. At move 12 in Vanishing Marks, what tension tier applies, how many meter segments are lit, and how many marks can be on the board?
10. Does Medium's 80 percent search branch imply an 80 percent win rate? Explain.
11. Why would drawing outgoing arcs only inside crack-slot make the effect difficult?
12. If the system requests reduced motion while Extreme is selected, what should happen to the storm effects?
13. In a minimax position, the AI maximizes and the opponent minimizes. If the opponent can choose scores 8, 2, or -4, which value is returned at that opponent node?
14. Why must classic() restore b[i] to an empty string after testing a candidate?
15. If you increase the match target from three to five, why is editing only scores[player] === 3 insufficient?

Answers and explanations

1. Center is 4, bottom-left is 6, and middle-right is 5.

2. ["X", "X", "", "", "O", "", "", "", ""]. It is O's turn because X just moved.

3. A player may complete a winning line on the final placement. The code checks a win before declaring a full-board draw.

4. The move is rejected. Square 0 is occupied at the time the legality check runs. A potentially expiring mark is not treated as an empty target.

5. X's queue becomes [2, 4, 8]. Square 0 becomes empty. There is no 0, 4, 8 win because 0 is gone.

6. Different sides may be next, or the queues may name different oldest marks. Those differences affect future behavior.

7. The score becomes 2 to 2. Nobody has three round wins yet, so the match continues.

8. clearTimeout cancels the stored callback; the revision ticket check rejects outdated callbacks. Other guards also check active status and turn ownership.

9. Critical tension applies. All nine segments are lit. At most six marks remain, three per side.

10. No. It is a probability of choosing the search branch on a decision. Opponent skill, position, and random moves that happen to be good also affect the result.

11. Each cell is clipped to its shape. A separate board-level overlay lets arcs extend beyond cell boundaries.

12. The preference override disables animated storm effects, stops timers, removes surge groups, and suppresses shaking. The selected option may still read Extreme.

13. -4, because this node represents an opponent choosing the worst result for the AI.

14. So the next candidate is tested from the same starting board. Otherwise hypothetical moves would accumulate incorrectly.

15. The interface also contains three pips, first-to-three labels, explanatory text, and match-completion wording. A robust change would define one target value and update all dependent behavior and tests.

30 Small coding exercises that build real understanding

Do these in a copied folder. The purpose is to practice tracing dependencies, not to modify the hearing copy immediately.

Exercise A Add an explicit round outcome label

Display "X won", "O won", or "Draw" alongside each round-log symbol. Find updateMatchUI(), where each list item is created. Change only the presentation. Confirm that scores and starter alternation remain unchanged.

This teaches the difference between changing how data is shown and changing the underlying outcome.

Exercise B Change the AI waiting time

Find the final delay argument in advanceTurn(). Change 380 to 700 for Solo. The AI search method should remain identical. Restart while it is waiting and verify that no old move appears.

This teaches that the thinking delay is presentation scheduling, not search depth.

Exercise C Make the match target configurable

Introduce a MATCH_TARGET constant. Use it for the completion comparison, pip count, and relevant text. Search for existing three-win phrases rather than editing one line and assuming the task is done.

A simple test should verify that reaching MATCH_TARGET - 1 wins does not finish the match and reaching MATCH_TARGET does.

Exercise D Improve the Vanishing heuristic

Think about whether a two-in-a-row containing the oldest mark should receive the same value as two newer marks. Write down the proposed rule before implementing it. Keep the rule engine unchanged.

Compare candidate decisions in controlled positions. Do not assume that increasing search depth or changing one weight automatically improves overall play.

Exercise E Save only user preferences

Use a copied version to experiment with localStorage for effects and sound preferences. Handle missing or invalid stored values. This would be a new feature; it is not currently implemented.

Do not describe localStorage as a server database. It is browser-side storage on that device and origin.

Exercise F Separate old and active styles

Use browser developer tools to inspect which .board-wrap animation rule actually wins. Identify an overridden older rule. Document why it is inactive before considering removal.

This teaches the cascade and prevents accidental deletion of a rule that still matters in another mode or media query.

31 A seven session study plan

Session	Focus	What you should be able to do afterward
1	Chapters 1 to 5	Describe the files and draw the board array
2	Chapters 6 to 10	Trace a click and explain oldest-mark removal
3	Chapters 11 and 12	Explain repetition, scores, and restarts
4	Chapters 13 to 16	Explain minimax, depth, pruning, and difficulty
5	Chapters 17 to 19	Explain timer safety, DOM updates, and CSS layers
6	Chapters 20 to 24	Explain how each major effect is generated
7	Chapters 25 to 30	Demonstrate the game and answer questions aloud

A session can be 30 to 60 minutes depending on what is new to you. Take extra time on move tracing and minimax instead of rushing to finish the schedule.

For each session, read a small part, close the guide, explain it aloud, and check your explanation. Then make one prediction and test it in the game. This is more useful than repeatedly reading the same paragraph without applying it.

One page worth of facts to remember

- The board is a nine-item array indexed 0 through 8.
- X and O are players; Classic and Vanishing are rulesets; Extreme is an effects setting.
- The engine decides legality and outcomes. The controller handles events, score, and presentation.
- A valid Vanishing move removes the mover's oldest mark before checking the win.
- An occupied oldest square is not a legal target.
- Eight fixed triples cover every possible winning line.
- Win checks come before draw checks.
- Repetition includes turn and queue order.
- A round can draw at three repeated positions or 60 moves.
- A match ends at three round wins; draws do not score.
- Classic search goes to terminal positions; Vanishing uses a depth limit and a heuristic.
- Easy and Medium mix random legal choices with search.
- Revision tickets and timer cancellation prevent stale AI callbacks.

- Lightning, magma, cracks, and shaking are cosmetic.
- Outward arcs use a separate SVG overlay with bounded cleanup.
- Testing provides evidence for specific behavior, not a guarantee of zero bugs.

32 Glossary in simple terms

Term	Meaning in this project
Algorithm	A defined procedure for solving a task
Alpha beta pruning	Skipping search branches that cannot improve a decision
API	A set of callable features, such as GridRules or Web Audio
Argument	A value passed into a function call
Array	An ordered list accessed by numbered indices
Assertion	A test statement that fails if an expected fact is false
Attribute	Extra information on an HTML or SVG element
Boolean	A true or false value
Callback	A function intended to run in response to an event or later action
Canvas	A browser drawing surface made of pixels
Class in CSS	A styling label on an element
Closure	A function remembering variables from its surrounding scope
Controller	Code coordinating inputs, rules, and interface updates
CSS cascade	The priority rules deciding which style declaration applies
DOM	The browser's object tree representing the page
Event	A reported action or change, such as a click
FIFO	First in, first out
Game state	The data describing the current situation
Game tree	Positions connected by possible moves
Guard	An early check that refuses inappropriate work
Heuristic	An estimate used when an exact result is not searched to completion
Immutable update	Producing a changed copy while preserving the earlier value
Index	A numbered position in an array

Term	Meaning in this project
Invariant	A property expected to remain true during valid operation
Key	A unique identifying representation for a lookup
Map	A collection associating keys with values
Memoization	Saving a computed result to reuse it
Minimax	Choosing against an opponent assumed to make the strongest reply
Mutation	Changing an existing object or array
Node	A point in a tree; also the Node.js runtime in another context
Parameter	A named input in a function definition
Ply	One move by one side
Procedural graphics	Visual content generated by instructions and numbers
Queue	An ordered collection where the earliest item leaves first
Recursion	A function calling itself on a smaller or later problem
Rendering	Converting state into visible output
Revision ticket	A saved version number used to reject outdated work
Set	A collection of distinct values
SVG	Graphics described as scalable shapes and paths
Terminal state	A finished position with a win or draw
Timeout	A scheduled callback after a requested delay
ViewBox	The coordinate space used by an SVG
Web Audio	Browser facilities for generating and processing sound
Web Worker	A way to run suitable work away from the page's main thread

33 What to claim and what not to claim

You can accurately explain the implemented rules, separate engine, search AI, local play modes, match progression, graphics techniques, and the checks that were actually run.

Do not describe Extreme as a stronger AI level. Do not say the tension meter measures winning probability. Do not say all three difficulty levels are unbeatable. Do not claim the game learns from every match.

Do not claim that 1,000 simulated games are a user study. Do not report a measured improvement in learning, engagement, or decision-making unless you conduct an appropriate evaluation. A intended benefit is not the same as a demonstrated result.

Do not say every line was written independently if AI tools helped generate it. Explain that assistance honestly and identify the work you personally performed. Being able to trace, test, and modify the implementation is more meaningful than memorizing a claim of authorship.

A useful answer when you have not measured something is: "I have not measured that yet. The current implementation does this, and I would test the claim by doing that." Replace "this" and "that" with a concrete behavior and method.

Your main preparation goal is to make the relationship between **input, state, rules, outcome, and display** clear. If you can demonstrate that relationship and answer follow-up questions honestly, you have a solid technical explanation of this project.

Appendix A Function reference and source map

These locations refer to the reviewed Storm Effects edition. Line numbers can change after an edit, so searching for a function name is usually the most reliable way to find it.

Public rules engine functions

Name	Responsibility
GridRules.create	Builds a new round and counts its starting position
GridRules.legal	Returns legal empty indices or no indices after a result
GridRules.play	Returns a new state for a legal move or null for an invalid move
GridRules.choose	Selects a legal computer move using difficulty and search
GridRules.key	Encodes turn, board, and queue order for repetition
GridRules.win	Returns the first matching winning triple or undefined
GridRules.lines	Exposes the eight winning index triples

Internal rules helpers

The internal other helper flips X and O. ordered returns legal indices in center, corner, edge order. evaluate estimates unfinished Vanishing positions. The nested classic function solves Classic positions using minimax and memoization. The nested search function uses depth-limited minimax and alpha-beta for Vanishing positions. They are not all exported as public GridRules methods.

Controller and effect functions

Function and line in script.js	Responsibility
restartAnimation at 36	Removes and reapplies an animation class after a layout read so the animation starts again.
initEmbers at 90	Creates the fixed pool of 16 ember elements and assigns visual variation.
initSmoke at 110	Creates the fixed pool of 12 smoke puffs and assigns visual variation.
generateNoiseTexture at 130	Paints a small pixel-noise image and exposes it as a CSS variable.
generateLavaTexture at 157	Paints the rock, hot spots, veins, and grain used by the magma layer.
playStamp at 252	Draws result text, starts its entrance animation, and schedules dissolution.
dissolveStamp at 272	Shuffles small canvas blocks and clears them over animation frames.
clearStamp at 309	Cancels stamp work and clears its canvas.

Function and line in script.js	Responsibility
isHumanControlled at 332	Checks whether the selected opponent mode gives a side to a human.
selectButtons at 335	Synchronizes selected-control styling and aria-pressed values.
renderMark at 379	Inserts one X or O SVG into a cell's mark-slot.
renderBoard at 382	Updates changed marks, state classes, accessibility labels, and board busy status.
makeMove at 401	Coordinates an accepted engine transition, result handling, rendering, and the next turn.
playerName at 442	Selects a display name such as You, AI, Player X, or AI O.
winMessage at 445	Builds the round-win sentence for the current opponent mode.
updateStatusText at 446	Displays whose turn it is and the relevant placement or expiry hint.
advanceTurn at 456	Schedules a guarded AI callback if the current side is computer-controlled.
updateMatchUI at 469	Updates scores, pip indicators, round number, and completed-round history.
sound at 490	Creates short Web Audio tones for moves, wins, or draws when enabled.
syncSoundButton at 507	Reflects the audio setting in the control's text and pressed state.
syncEffects at 513	Applies the selected effects level and system motion override, restarting or stopping effects as appropriate.
tensionTier at 536	Maps move count to calm, mid, high, or critical.
updateTensionEffects at 543	Updates meter segments and tension classes, then starts or stops storm scheduling.
resetTensionEffects at 577	Removes active tension classes and stops lightning and surges.
svgElement at 597	Creates an SVG element in the correct namespace and assigns attributes.
stopSurges at 602	Cancels surge scheduling and cleanup timers and removes all active surge groups.
scheduleSurge at 607	Schedules recurring bursts from existing crack vents without starting duplicate loops.
emitSurge at 618	Builds one bounded temporary group of outward arcs, vent rings, and flying particles.
startLightning at 674	Starts a storm with an immediate strike and schedules future strikes and surges.
scheduleLightning at 681	Chooses the next strike delay and rechecks state when the callback runs.
lightningTier at 699	Reports whether move count is in a tier that allows lightning.

Function and line in script.js	Responsibility
stopLightning at 704	Cancels lightning and surge scheduling and removes active strike and jolt classes.
strikeLightning at 713	Restarts the ambient strike and arena jolt before targeting a cell.
strikeBoltOnBoard at 721	Measures a target cell and populates the screen-space main and branch paths.
impactCell at 751	Replaces a tile's fractures, stores vents, applies impact classes, and emits a burst.
clearCracks at 773	Clears vent data, removes stored fracture artwork, and resets cracked-cell tracking.
buildCrackSvg at 784	Returns local fracture shapes, center coordinates, and vent endpoints.
pickStrikeTarget at 827	Prefers a not-yet-cracked tile, otherwise chooses any tile.
buildBoltPath at 837	Creates the main jagged path that converges onto a measured target.
buildBranches at 862	Creates two short forks from intermediate points on the main bolt.
cellCenterPercent at 885	Converts a tile center into percentages of the board wrapper.
drawWinLine at 891	Uses the winning triple to draw a line through its first and last cells.
triggerWinEffects at 906	Stops tension effects and starts winner-colored result presentation.
triggerDrawEffect at 926	Stops tension effects and shows the neutral draw result.
spawnSparks at 933	Creates short-lived celebration particles with event and timer cleanup.
newMatch at 966	Resets scores and match metadata, then starts the first round.
restartGame at 971	Cancels old round work, creates the selected ruleset's initial state, and refreshes the interface.

Event listeners to find

The opponent, difficulty, and ruleset buttons call newMatch after a genuine selection change. Cell click listeners route human input into makeMove. Cell keydown listeners handle arrows. Sound and effect listeners update preferences. The visibilitychange listener cancels or resumes appropriate scheduling. The main action button chooses restart, next round, or rematch. The New match button calls newMatch. A window resize listener redraws an existing winning line.

Main DOM connections

HTML identifier or class	Connected feature
board and .cell	Nine input buttons and turn styling
status and turnHint	Turn and result messages

HTML identifier or class	Connected feature
scoreX and scoreO	Numeric match scores
pipsX and pipsO	Progress toward three wins
roundLabel and roundHistory	Round number and completed outcomes
modeTabs and difficultyTabs	Opponent and computer strength
ruleTabs	Classic or Vanishing selection
restart and resetScore	Contextual round action and New match
effectsSelect and motionNote	Intensity choice and reduced-motion explanation
tensionMeter and moveLabel	Visual buildup and numeric placement count
winLine	Winner line geometry
boltSvg and its paths	Viewport lightning geometry
fxStamp	Canvas result text
.surge-layer	Dynamically inserted outgoing electricity SVG

Appendix B Read the exact rules engine

The following is the complete reviewed game-core.js. Read it in this order: create and legal, then play, then win and key, then choose and its search functions. The explanations in Chapters 8 through 16 supply the meaning behind the compact expressions. Long lines may wrap for reading; the playable source file remains the authoritative copy for editing.

```
/* Pure rules and AI. No DOM, timers, or external dependencies. */
(function (root) {
  'use strict';
  const lines = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]];
  const other = p => p === 'X' ? 'O' : 'X';
  const win = (b,p) => lines.find(line => line.every(i => b[i] === p));
  const key = s => s.turn + ':' + s.board.map(v => v || '-').join('') + ':' +
      s.queues.X.join('') + ':' + s.queues.O.join('');
  function create(rule = 'classic', starter = 'X') {
    const s = { rule, turn: starter, board: Array(9).fill(''), queues: { X: [], O: [] },
        moves: 0, winner: null, draw: null, line: null, seen: {} };
    s.seen[key(s)] = 1;
    return s;
  }
  const legal = s => s.winner || s.draw ? [] : s.board.flatMap((v,i) => v ? [] : [i]);
  function play(s,i) {
    if (!Number.isInteger(i) || !legal(s).includes(i)) return null;
    const n = { ...s, board: [...s.board], queues: { X: [...s.queues.X], O:
        [...s.queues.O] }, seen: {...s.seen}, moves: s.moves + 1 };
    const p = s.turn;
    // Select an empty tile, then remove the oldest mark before checking victory.
    if (s.rule === 'vanish' && n.queues[p].length === 3) n.board[n.queues[p].shift()] =
        '';
    n.board[i] = p;
    n.queues[p].push(i);
    n.line = win(n.board,p) || null;
    n.turn = other(p);
    if (n.line) n.winner = p;
    else if (n.board.every(Boolean)) n.draw = 'Board full';
    else {
      const k = key(n);
      n.seen[k] = (n.seen[k] || 0) + 1;
      if (n.seen[k] >= 3) n.draw = 'Position repeated three times';
      else if (n.moves >= 60) n.draw = '60-move limit reached';
    }
    return n;
  }
  function evaluate(s,p) {
    let score = 0;
    for (const line of lines) {
      const mine = line.filter(i => s.board[i] === p).length;
      const theirs = line.filter(i => s.board[i] === other(p)).length;
      if (!theirs) score += [0,1,7,100][mine];
      if (!mine) score -= [0,1,7,100][theirs];
    }
    return score;
  }
  const ordered = s => [4,0,2,6,8,1,3,5,7].filter(i => !s.board[i]);
  function choose(s,difficulty = 'hard',random = Math.random) {
    const moves = legal(s);
    if (!moves.length) return null;
    const pick = () => moves[Math.min(moves.length - 1,Math.floor(random() *
```

```
        moves.length))];
    if (difficulty === 'easy' && random() < .7) return pick();
    if (difficulty === 'medium' && random() < .2) return pick();
    const player = s.turn;
    const depth = s.rule === 'classic' ? 9 : difficulty === 'hard' ? 8 : 4;
    const cache = new Map();
    function classic(b,turn,left) {
      if (win(b,player)) return 1000 + left;
      if (win(b,other(player))) return -1000 - left;
      if (b.every(Boolean)) return 0;
      const k = b.map(v => v || '-').join('') + turn;
      if (cache.has(k)) return cache.get(k);
      let best = turn === player ? -Infinity : Infinity;
      for (let i = 0; i < 9; i++) if (!b[i]) {
        b[i] = turn;
        const value = classic(b,other(turn),left - 1);
        b[i] = '';
        best = turn === player ? Math.max(best,value) : Math.min(best,value);
      }
      cache.set(k,best);
      return best;
    }
    function search(position,left,alpha,beta) {
      if (position.winner) return position.winner === player ? 1000 + left : -1000 -
          left;
      if (position.draw) return 0;
      if (!left) return evaluate(position,player);
      const max = position.turn === player;
      let best = max ? -Infinity : Infinity;
      for (const i of ordered(position)) {
        const value = search(play(position,i),left - 1,alpha,beta);
        best = max ? Math.max(best,value) : Math.min(best,value);
        if (max) alpha = Math.max(alpha,best); else beta = Math.min(beta,best);
        if (beta <= alpha) break;
      }
      return best;
    }
    let best = -Infinity, result = moves[0];
    for (const i of ordered(s)) {
      const n = play(s,i);
      const value = s.rule === 'classic' ? classic([...n.board],n.turn,depth - 1) :
          search(n,depth - 1,-Infinity,Infinity);
      if (value > best) { best = value; result = i; }
    }
    return result;
  }
  const api = { create,play,legal,choose,key,win,lines };
  if (typeof module !== 'undefined' && module.exports) module.exports = api;
  else root.GridRules = api;
})(typeof globalThis !== 'undefined' ? globalThis : this);
```